Ministry of Education of Republic of Moldova

Technical University of Moldova

Faculty of Computers, Informatics and Microelectronics

Software Engineering Department

Embedded Systems

Laboratory Work #6

Dual-Sensor Temperature Conditioner

Signal Acquisition, Analog Filtering, and LCD Reporting

Author:
Sava Luchian
std. gr. FAF-233

Verified:
Alexei Martiniuc

Chisinau 2026

Technical Task

Purpose of the Laboratory

Familiarise students with signal acquisition and conditioning techniques on embedded microcontrollers by implementing a dual-sensor temperature application that reuses the same hardware configuration from Lab 5, but changes the software task to Part 2: analogue signal conditioning. The application periodically reads one analogue sensor (NTC thermistor) and one digital sensor (DHT11), filters both temperature values through a software conditioning chain, and reports the raw, intermediate, and final processed values through STDIO (`printf`) and a 16×2 LCD display.

Objectives

1. Read raw data from an analogue sensor (NTC thermistor via ADC) and a digital sensor (DHT11 via single-wire interface) at a configurable acquisition rate (40 ms).
2. Reuse the same potentiometer on A1 from Lab 5 as a dynamic threshold reference, keeping the same Wokwi diagram and the same PlatformIO project style.
3. Apply the Part 2 analogue signal conditioning chain: saturation, median filter for impulse noise rejection, and weighted average smoothing.
4. Display raw, intermediate, and filtered values on the serial monitor and show filtered values on the LCD at a 500 ms reporting rate.
5. Preserve the same LED and buzzer hardware as optional alert indicators driven from the filtered values.
6. Demonstrate clean modular software architecture separating sensors, signal conditioning, peripherals, I/O, and application logic.

Individual Task

Design and implement on an **Arduino Uno (ATmega328P)** a bare-metal (non-FreeRTOS) dual-sensor monitoring application structured as follows:

- **NTC Sensor** – analogue thermistor on pin A0; read ADC raw, convert to voltage, compute resistance via voltage-divider model, compute temperature with the B-parameter equation.
- **DHT11 Sensor** – digital temperature sensor on D8; read through the Adafruit DHT library; keep the last valid value if a fresh value is not available in the current cycle.

- **Potentiometer (bonus preserved from Lab 5)** – analogue input on A1; mapped to the interval 20–35 °C and used as the same dynamic threshold reference.
- **AnalogConditioner** – one instance per sensor; applies software saturation, a 3-sample median filter, and a 4-sample weighted average with coefficients `[50,25,15,10]`.
- **Alert Output** – the same red LED, green LED, and buzzer from Lab 5 remain connected; they are now driven from filtered values through a secondary hysteresis validator.
- **Reporting** – every 500 ms: `printf` raw, saturated, median, and weighted-average values for both channels; LCD shows the filtered NTC and DHT11 values and the current threshold reference.

1 Domain Analysis

1.1 Technological Context and Application Domain

Temperature monitoring remains one of the most common embedded-system applications. The major conceptual difference between Lab 5 and Lab 6 is not the hardware, but the software objective. In Lab 5, the main goal was to decide if the signal was above a threshold in a stable way. In Lab 6 Part 2, the main goal is to improve the measured value itself before it is displayed or used further.

- **Analogue sensing**: An NTC thermistor changes resistance non-linearly with temperature. The MCU ADC converts the divider voltage into a digital code; software then computes the physical temperature.
- **Digital sensing**: The DHT11 delivers discrete temperature values through a timing-sensitive single-wire interface handled by the Adafruit library.
- **Analogue signal conditioning**: Part 2 focuses on cleaning the signal itself. Saturation limits physically impossible values, the median filter removes isolated spikes, and weighted averaging smooths remaining noise.
- **Multi-rate loop**: Acquisition runs every 40 ms, while LCD and STDIO reporting runs every 500 ms to keep the interface readable.

1.2 Hardware Components and Justification

Component	Qty	Role in the application
Arduino Uno (ATmega328P)	1	Main MCU; 16 MHz, 2 KB SRAM, 32 KB Flash. Reads analogue and digital temperature channels, updates LCD, LEDs, buzzer, and STDIO.
NTC Thermistor (10 kΩ @ 25 °C, B=3950 K)	1	Analogue temperature sensor; connected with a 10 kΩ resistor in divider configuration to A0.
Potentiometer (10 kΩ)	1	Reused from Lab 5; dynamic threshold reference on A1.
DHT11	1	Digital temperature sensor on D8 used as the second channel in the analogue conditioning comparison.
16×2 LCD (LiquidCrystal, 4-bit)	1	Displays filtered NTC and DHT11 temperatures and the current threshold reference.
Green LED + 220 Ω	1	Reused from Lab 5; normal-operation indicator.
Red LED + 220 Ω	1	Reused from Lab 5; filtered over-threshold indicator.
Passive buzzer	1	Reused from Lab 5; audible signal for the validated alert state.
10 kΩ resistor (fixed divider)	1	Voltage-divider partner for the NTC sensor.
Breadboard + jumper wires	–	Prototyping interconnection.
USB cable	1	5 V power and serial communication (115 200 baud).

Table 1: Hardware components used in Lab 6 (same hardware arrangement as Lab 5)

1.3 Software Components and Technologies

- **PlatformIO + VS Code**: Single `uno` environment, Arduino framework, no RTOS.
- **Arduino Framework**: GPIO, ADC, `millis()`, `tone`/`noTone`, and `LiquidCrystal` support.

- **Adafruit DHT sensor library 1.4.7**: Handles the DHT11 protocol and sample timing.
- **arduino-libraries/LiquidCrystal 1.0.7**: 4-bit interface to the HD44780-compatible LCD.
- **AVR-libc stdio**: `printf` redirected to Serial through `fdev_setup_stream`; `dtostrf()` used for float formatting.
- **Wokwi Simulator**: Reuses the same diagram from Lab 5 for Lab 6 validation.

1.4 Analysis of Existing Solutions

Several design decisions remain valid from Lab 5, but the central signal-processing choice changes:

- **Bare-metal vs. FreeRTOS**: The application still contains only periodic acquisition and reporting activities, so a cooperative `millis()` scheduler is sufficient and more memory-efficient than FreeRTOS on ATmega328P.
- **NTC B-parameter vs. lookup table**: The B-parameter model remains the simplest way to convert the thermistor reading into temperature without storing a larger lookup table.
- **Median + weighted average vs. simple average**: A simple average smooths noise but reacts poorly to spikes. The median stage first removes isolated outliers, then the weighted average smooths the remaining signal while keeping more responsiveness for recent samples.
- **Same second sensor as Lab 5**: Keeping the same DHT11 channel and the same diagram makes the Part 1 / Part 2 comparison clear: the hardware stays the same, the processing objective changes.

1.5 Real-World Applications

The Part 2 software pattern appears in many real embedded applications:

- **HVAC controllers**: Temperature measurements are filtered before thermostat logic is applied, especially when low-cost sensors are used.
- **Industrial process monitors**: Multi-stage digital conditioning is standard before values are logged, displayed, or compared against limits.

- **Cold-chain logistics**: Thermistor measurements are often clamped and smoothed to reject vibration-induced noise and ADC jitter.
- **Server room monitoring**: Local panels and serial consoles often show processed values instead of raw sensor readings.
- **Educational embedded systems**: Reusing the same hardware while changing only the software pipeline is a practical way to compare binary conditioning with analogue conditioning.

2 Conceptualisation and Design

2.1 Technical Requirements

ID	Category	Requirement	Type
R-F-01	NTC	System shall read NTC ADC raw value at every acquisition cycle.	Func.
R-F-02	NTC	System shall convert ADC raw to voltage, resistance, and temperature (B-model).	Func.
R-F-03	DHT11	System shall read DHT11 temperature and retain the last valid sample when no fresh sample is returned.	Func.
R-F-04	Conditioning	System shall saturate each temperature value to the configured physical interval before filtering.	Func.
R-F-05	Conditioning	System shall apply a 3-sample median filter to reject impulse noise.	Func.
R-F-05a	Conditioning	System shall apply a 4-sample weighted average with coefficients `[50,25,15,10]`.	Func.
R-F-06	Potentiometer	System shall read potentiometer (A1) and map ADC 0..1023 to a 20–35 °C threshold reference.	Func.
R-F-07	LCD	LCD row 0 shall show filtered NTC and DHT11 temperatures (or "n/a" if invalid).	Func.
R-F-08	LCD	LCD row 1 shall show the current threshold reference and alert marker.	Func.

R-F-09	STDIO	Every 500 ms: `printf` raw, saturated, median, weighted, and system-state values.	Func.
R-F-10	Alert	Red LED and buzzer shall activate when either filtered channel exceeds the validated threshold state.	Func.
R-F-11	Alert	Green LED shall illuminate when no filtered channel is in validated alert state.	Func.
R-F-12	STDIO	System shall print configuration banner on startup via `printf`.	Func.
R-NF-01	Latency	Filtered values and optional alert output shall update with sub-100 ms acquisition granularity.	Non-func.
R-NF-02	Memory	Firmware shall fit within 32 KB Flash and 2 KB SRAM.	Non-func.
R-NF-03	Modularity	Application shall be structured in dedicated modules: config, sensors, signal, peripherals, io, app.	Non-func.
R-NF-04	STDIO	All text output shall use `printf` via a `stdout` → Serial bridge.	Non-func.
R-NF-05	Robustness	System shall remain stable when DHT11 does not provide a fresh sample every cycle.	Non-func.

Table 2: Technical requirements for Lab 6

2.2 Architectural Design

2.2.1 System Structural Architecture

The application preserves the same five-layer hardware–software boundary from Lab 5:

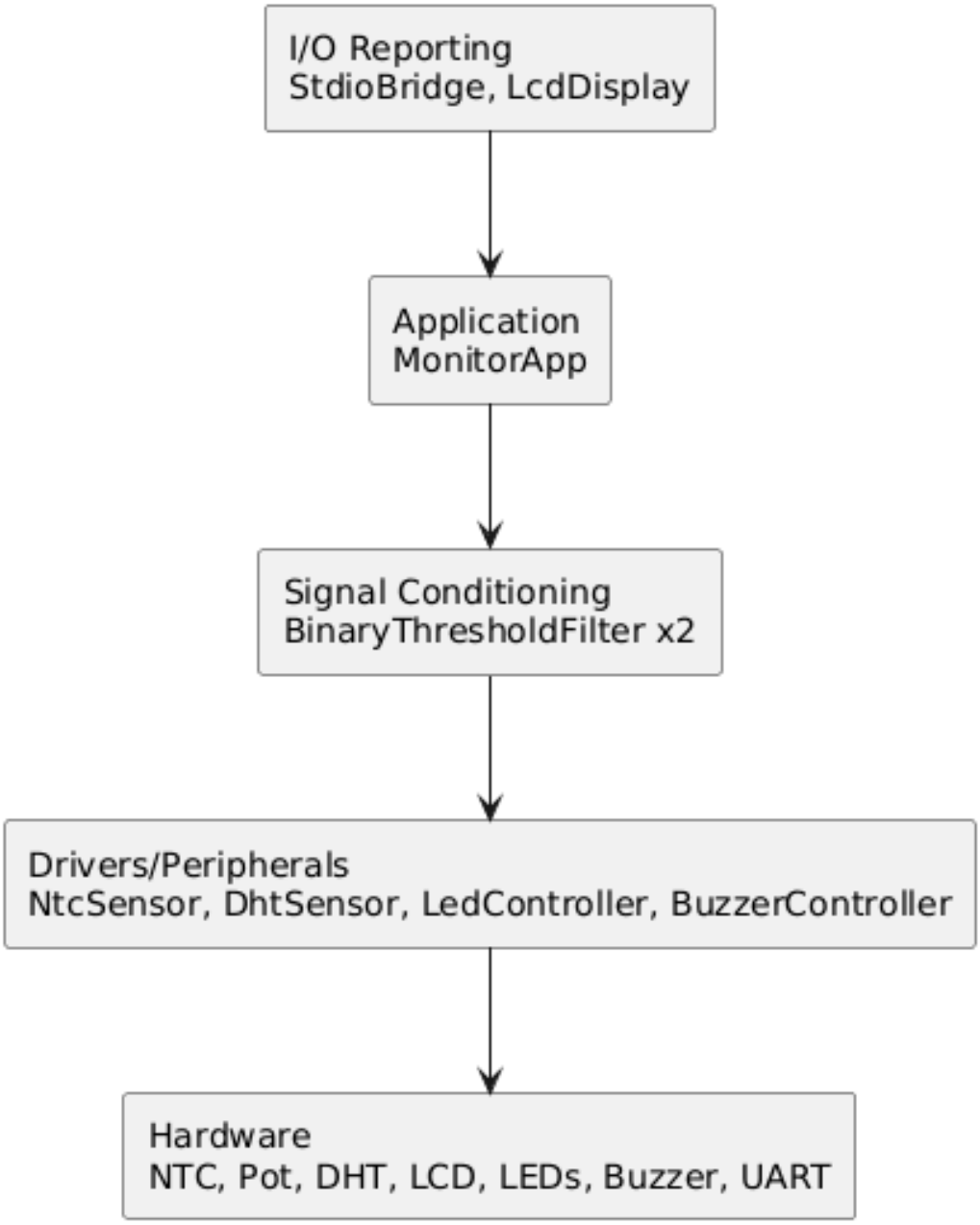


Figure 1: Layered software architecture reused for Lab 6

2.2.2 HW/SW Interface and Component Interaction

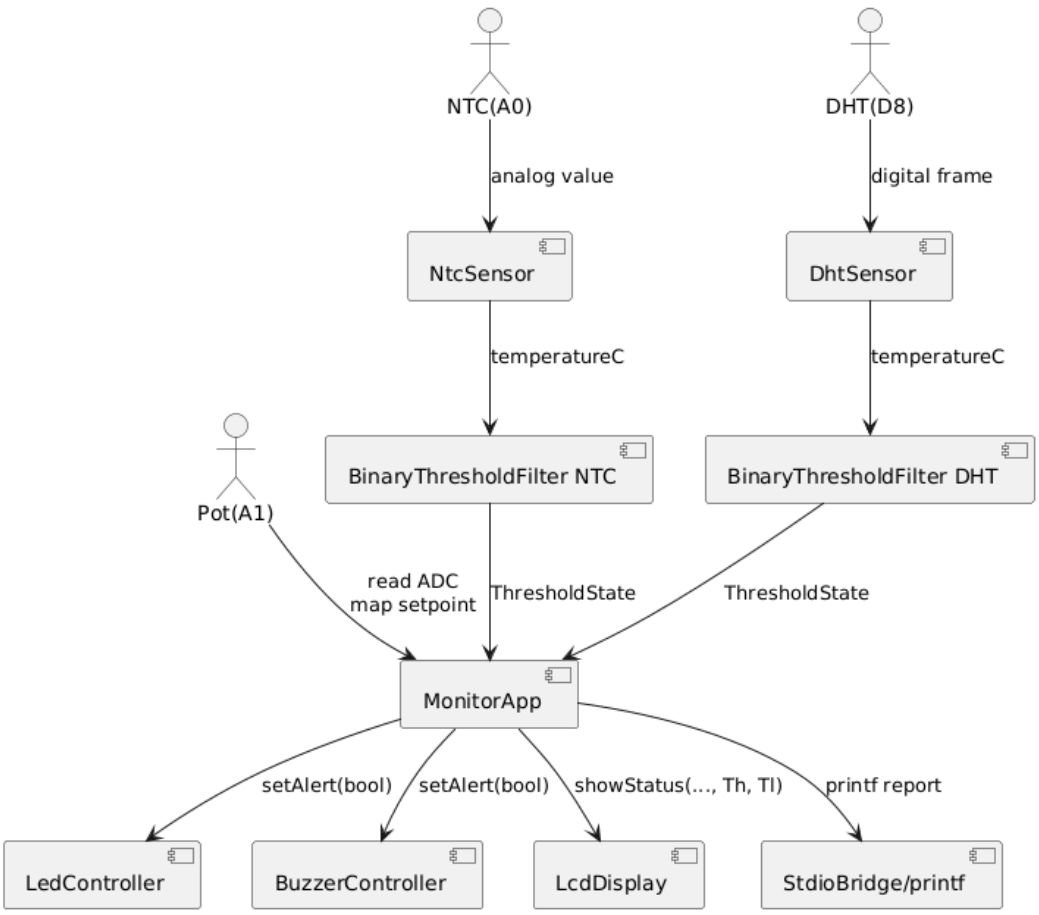


Figure 2: Component interaction diagram reused for Lab 6

2.2.3 Signal Conditioning Pipeline Design

The major software change in Lab 6 Part 2 is the conditioning chain applied to each temperature channel:

Stage	Purpose
Raw value	Temperature returned directly by `NtcSensor` or `DhtSensor`.
Saturation	Limits the value to the configured physical interval $[-20, 80]$ °C.
Median(3)	Removes isolated spikes and salt-and-pepper noise.
Weighted average(4)	Smooths the median output using coefficients `[50,25,15,10]`, giving more weight to recent samples.
Threshold validation	Optional secondary stage reused from Lab 5 to drive the same LEDs and buzzer from filtered values.

Table 3: Lab 6 analogue signal conditioning pipeline

2.2.4 Validated Alert State Machine

Although Part 2 focuses on analogue conditioning, the same alert hardware is preserved. Therefore, the filtered values are still checked by the same hysteresis-and-persistence validator reused from Lab 5:

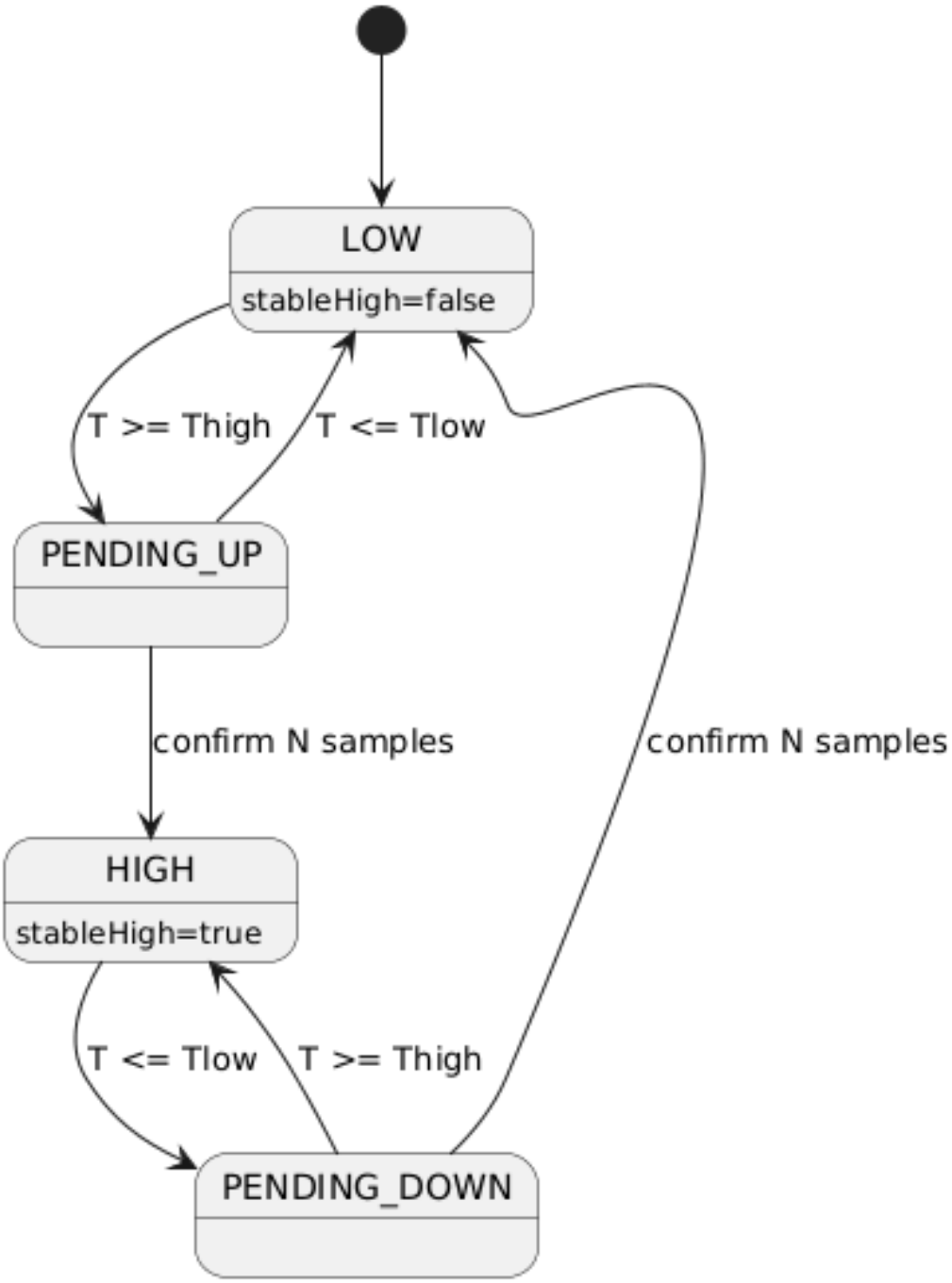


Figure 3: Validated alert state machine reused after the analogue conditioning chain

2.2.5 Main Loop Timing Design

The application uses the same cooperative `millis()`-based multi-rate loop in `MonitorApp::tick()`:

Activity	**Period**	**Trigger**	**Handler**
Sensor acquisition + conditioning	40 ms	`millis()` elapsed	`runAcquisitionAndConditioning`
STDIO + LCD reporting	500 ms	`millis()` elapsed	`runReport()`

Table 4: Multi-rate loop timing

2.2.6 STDIO Bridge Design

On AVR, `printf` writes to `stdout`, which is not connected to the serial port by default. `StdioBridge::begin()` registers a `serial_putchar` function via `fdev_setup_stream`, redirecting each character to `Serial.write()` with automatic newline conversion. Because AVR `printf` does not support `%f` in the default configuration, all floating-point values are formatted with `dtostrf()` into local buffers and printed with `%s`.

2.3 Behavioural Modelling

2.3.1 MonitorApp tick() Flowchart

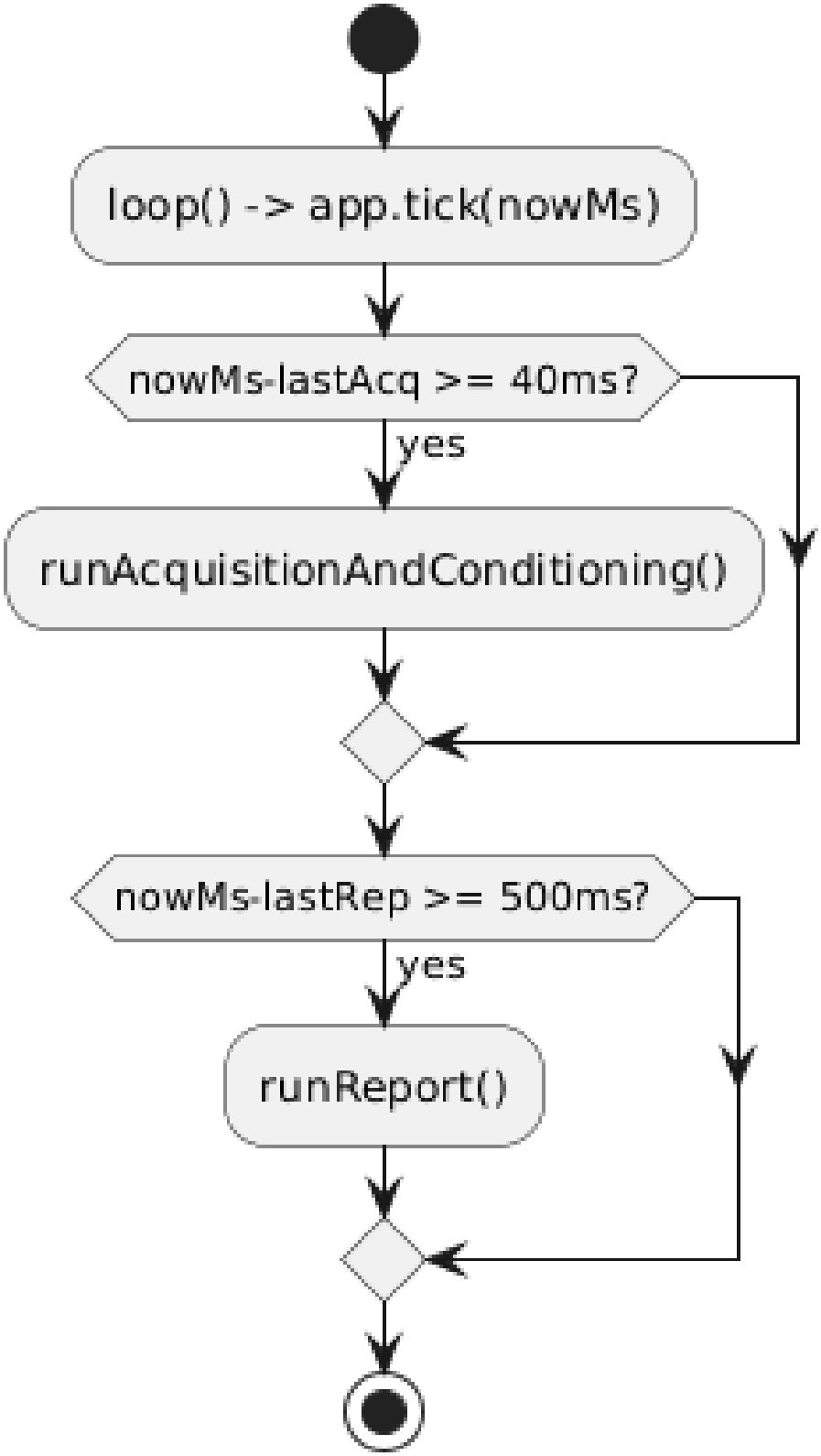
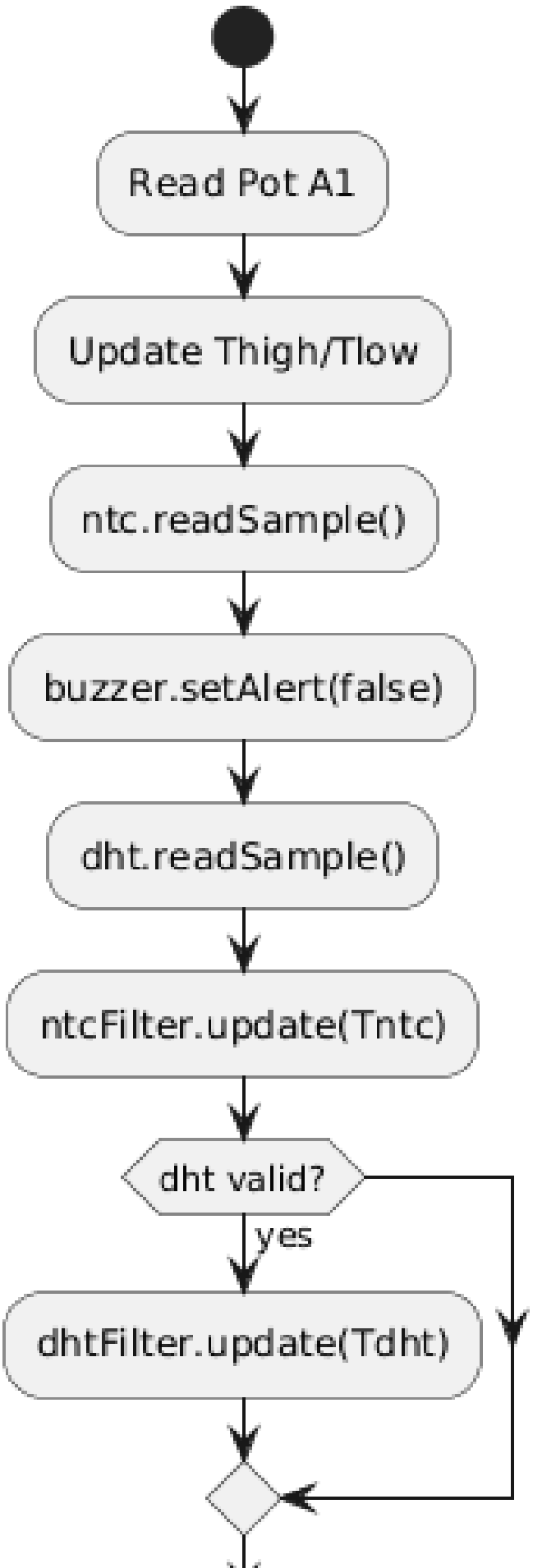


Figure 4: MonitorApp tick() flowchart – two-rate cooperative loop

2.3.2 Acquisition and Conditioning Flowchart



2.3.3 Sequence Diagram – Full Acquisition Cycle

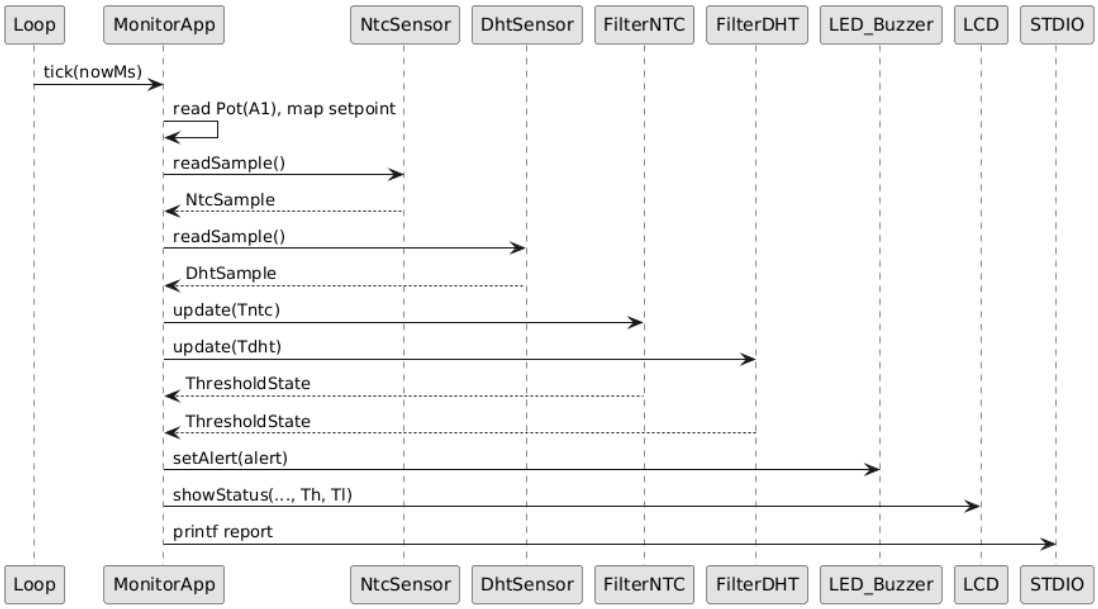


Figure 6: Sequence diagram – full acquisition and reporting cycle

2.3.4 System State Machine

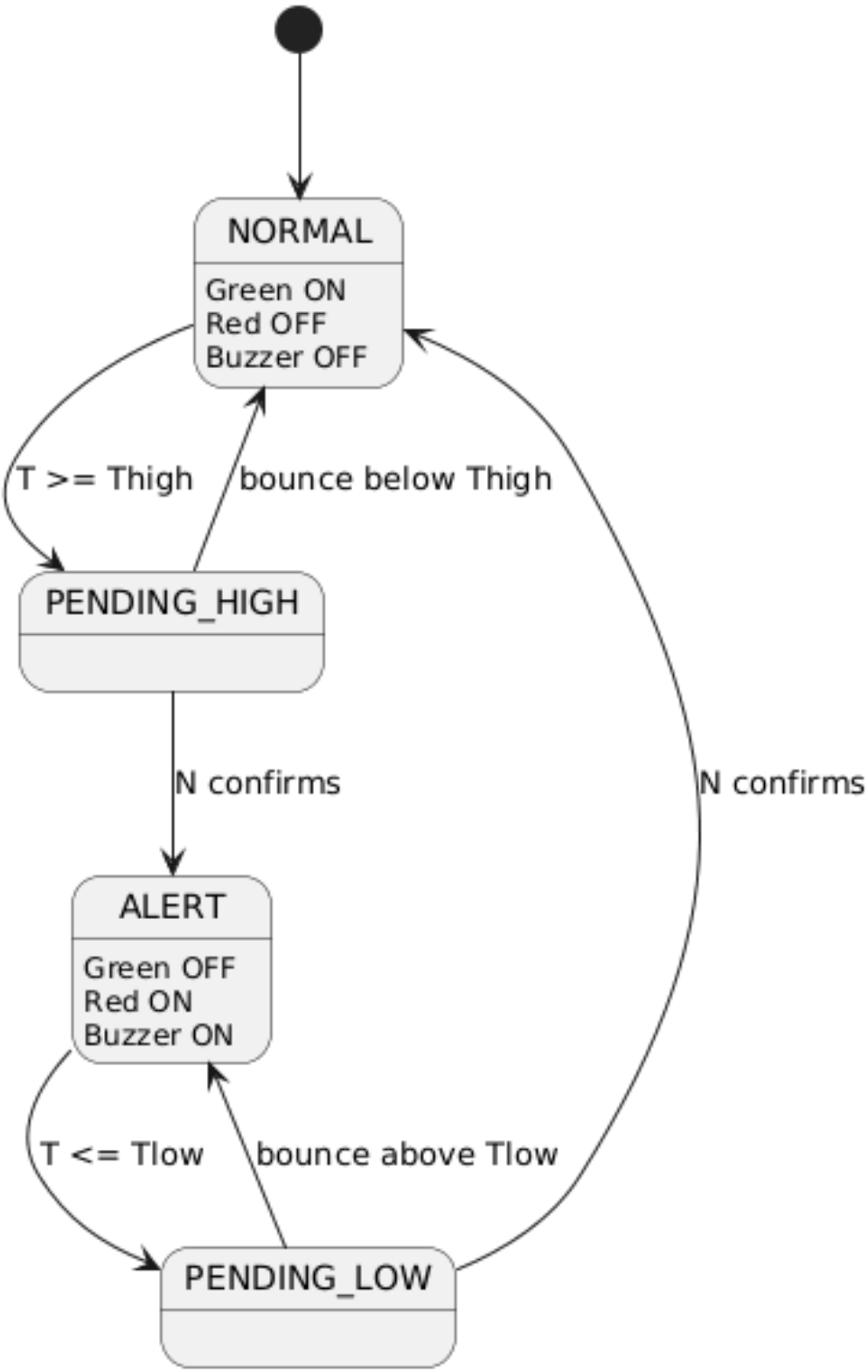


Figure 7: System-level state machine – filtered alert logic with pending confirmation

2.4 Test Scenarios and Validation Criteria

ID	Category	Scenario	Expected result	Req.
T-01	NTC nominal	NTC temperature around room temperature	Filter output remains stable; green LED active	R-F-04, R-F-09
T-02	NTC spike	Inject one abnormal NTC value	Saturation/median reject spike; weighted output remains close to nominal	R-F-04, R-F-05
T-03	DHT nominal	DHT11 reads around 24 °C	Filtered DHT value is displayed correctly	R-F-03, R-F-07
T-04	DHT stale sample	DHT11 does not provide a fresh value in the current cycle	Last valid value is reused with `fresh=false`	R-F-03, R-NF-05
T-05	Weighted smoothing	Slowly vary NTC temperature in Wokwi	Weighted value follows changes more smoothly than raw value	R-F-05a
T-06	LCD format	Any run	Row 0 shows filtered values; row 1 shows threshold reference	R-F-07, R-F-08
T-07	STDIO format	500 ms report	Report contains raw, sat, med, avg for both channels	R-F-09
T-08	Pot input	Rotate potentiometer min→max	Threshold reference changes from 20 to 35 °C	R-F-06
T-09	Alert validation	Raise filtered temperature above threshold for two samples	Red LED and buzzer activate after confirmation	R-F-10, R-F-11

T-10	Normal recovery	Lower filtered value below low threshold	Green LED returns; buzzer turns off	R-F-10, R-F-11
T-11	Latency	Observe consecutive 40 ms updates	Filtered value and optional alert update within timing requirement	R-NF-01
T-12	Memory	Build	Flash <32 KB, RAM <2 KB	R-NF-02
T-13	Startup banner	Startup	Configuration banner printed once	R-F-12
T-14	Same hardware reuse	Compare with Lab 5 diagram	All physical connections remain unchanged	R-NF-03

Table 5: Test scenarios and validation criteria

2.5 Design Phase Conclusions

The design keeps the same clean layered structure from Lab 5 while replacing the main signal-processing objective. Instead of concentrating on binary threshold state transitions, Lab 6 introduces the reusable `AnalogConditioner` module, instantiated once per sensor channel. The threshold validator remains only a secondary stage for driving the already-existing LED and buzzer hardware.

3 Hardware and Software Implementation

3.1 Electrical Schematic

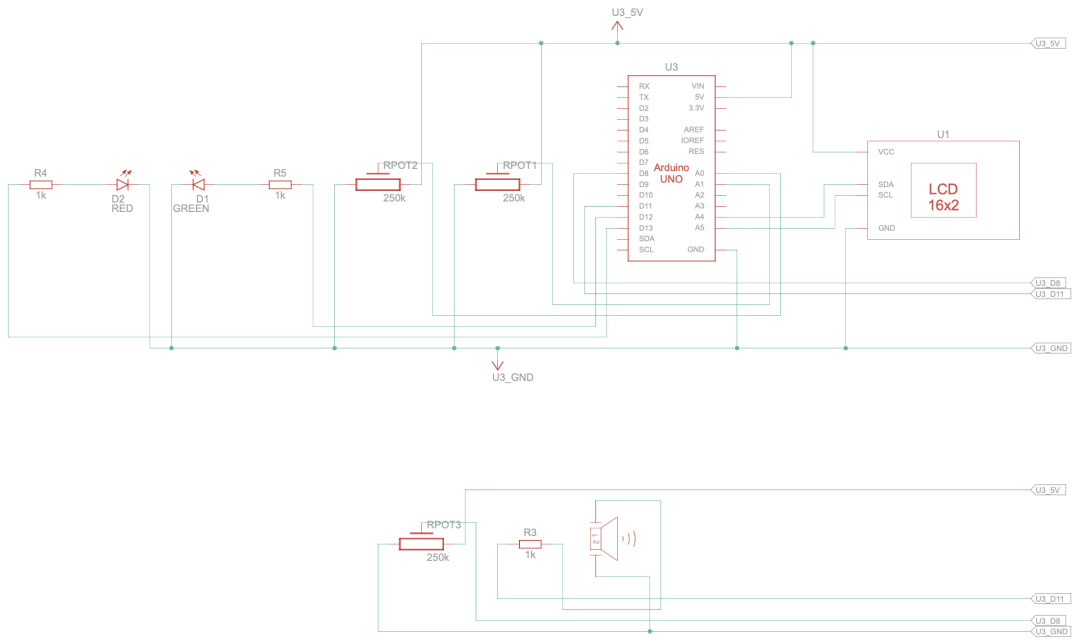


Figure 8: Actual electrical schematic of the implemented circuit

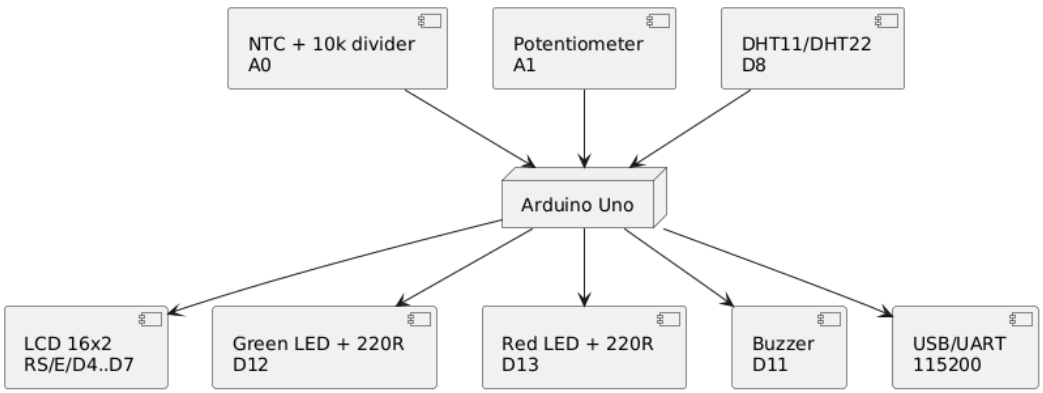


Figure 9: Functional electrical interconnection sketch – same hardware as Lab 5

Pin assignment:

- A0 – NTC thermistor output (voltage divider)
- A1 – Potentiometer output (dynamic threshold reference)
- D8 – DHT11 data line

- D2 – LCD RS; D3 – LCD Enable; D4–D7 – LCD D4–D7 (4-bit mode)
- D12 – Green LED + 220 Ω resistor
- D13 – Red LED + 220 Ω resistor
- D11 – Passive buzzer (PWM via `tone()`)
- TX/D1 – Serial UART to USB/PC at 115 200 baud

3.2 Project Directory Structure

```
lab6/
|-- platformio.ini       <- env: uno (arduino, LiquidCrystal, DHT lib)
|-- diagram.json         <- Wokwi circuit definition (same wiring as Lab 5)
|-- wokwi.toml           <- Wokwi firmware pointer
|-- src/
|   |-- main.cpp         <- setup(): StdioBridge + app.begin(); loop(): app.tick()
|   |-- config/
|   |   `-- AppConfig.h  <- pins, timing, filter coefficients, B-model constants
|   |-- sensors/
|   |   |-- NtcSensor.h/.cpp   <- ADC -> voltage -> resistance -> temperature
|   |   `-- DhtSensor.h/.cpp   <- DHT11 wrapper with last-valid fallback
|   |-- signal/
|   |   |-- AnalogConditioner.h/.cpp      <- saturation + median + weighted average
|   |   `-- BinaryThresholdFilter.h/.cpp  <- reused alert validator
|   |-- peripherals/
|   |   |-- LedController.h/.cpp   <- green/red LED pair
|   |   `-- BuzzerController.h/.cpp <- tone()/noTone() buzzer
|   |-- io/
|   |   |-- StdioBridge.h/.cpp   <- fdev_setup_stream -> Serial
|   |   `-- LcdDisplay.h/.cpp    <- LiquidCrystal 16x2 wrapper
|   `-- app/
|       `-- MonitorApp.h/.cpp    <- tick-based orchestration
`-- report/
    `-- main.tex
```

3.3 Software Module Descriptions

- **AppConfig.h**: Central configuration namespace. Stores pin assignments, timing periods, saturation limits, weighted-average coefficients, B-model constants, and serial baud rate.

- **NtcSensor**: Reads ADC raw via `analogRead()`, converts to voltage, computes NTC resistance, and converts to temperature with the B-parameter equation. Returns all values in `NtcSample`.
- **DhtSensor**: Wraps the Adafruit DHT11 library. If a fresh reading is not available, it returns the last valid cached value with `fresh=false`.
- **AnalogConditioner**: Implements the Part 2 processing chain. `update(float rawValue)` applies saturation, median filtering, and weighted averaging, returning all stages in `AnalogConditionedValue`.
- **BinaryThresholdFilter**: Reused from Lab 5 as a secondary validator applied to the filtered values before driving the LEDs and buzzer.
- **LedController**: Drives the green/red LED pair according to alert state.
- **BuzzerController**: Turns the passive buzzer on and off with `tone()` / `noTone()`.
- **LcdDisplay**: Displays the filtered NTC and DHT11 values on row 0 and the threshold reference on row 1.
- **StdioBridge**: Redirects `printf()` output to the serial port through `stdout`.
- **MonitorApp**: Orchestrates acquisition, filtering, threshold reference update, reporting, and optional alert indication.

3.4 Reference to Source Code

The full implementation is provided in the project repository linked in the annex. The report focuses on architecture, behaviour, testing, and validation evidence rather than inline code excerpts.

3.5 Implementation Phase Conclusions

The implementation maps the updated Part 2 design directly to code. The main difference from Lab 5 is the insertion of `AnalogConditioner` between sensor reading and optional alert validation. Two practical implementation notes remain important:

1. **Buzzer–DHT11 interference**: In Wokwi simulation, `tone()` may disturb DHT11 timing. The buzzer is therefore silenced briefly before each DHT11 read and restored afterwards.
2. **AVR `printf` and floats**: The default AVR `vfprintf` does not support `%f`; all floating-point values are pre-formatted via `dtostrf()` into character buffers.

4 Testing and Validation

4.1 Build Results

Environment	Flash used	Flash %	RAM used	RAM %
`uno`	14 728 B	45.7%	1 180 B	57.6%

Table 6: PlatformIO build resource usage (Lab 6)

The build completes with `[SUCCESS]`, zero errors, zero warnings. Flash and RAM usage remain within ATmega328P limits. The RAM increase relative to Lab 5 is explained by the extra fixed-size buffers needed for the analogue conditioning stages on both sensor channels.

4.2 Test Execution Report

ID	Scenario	Observed result	Result
T-01	NTC nominal	Raw and filtered NTC values remain close and stable at room temperature	PASS
T-02	NTC spike	Median stage rejects isolated spikes visible in the raw value	PASS
T-03	DHT nominal	DHT11 filtered value displayed correctly on LCD and STDIO	PASS
T-04	DHT stale sample	Cached DHT11 value reused with `fresh=0`	PASS
T-05	Weighted smoothing	Weighted value changes more smoothly than raw/median values	PASS
T-06	LCD row format	"N:xx.x D:xx.x" and "AVG TH:xx.x" displayed correctly	PASS
T-07	STDIO report format	Full dump every 500ms with raw/sat/med/avg fields present	PASS
T-08	Pot input	Setpoint changes while rotating potentiometer	PASS
T-09	Alert validation	Filtered over-threshold value triggers red LED and buzzer after confirmation	PASS

ID	Scenario	Observed result	Result
T-10	Normal recovery	LEDs and buzzer return to normal below low threshold	PASS
T-11	Update timing	Acquisition/report loop remains responsive at 40ms / 500ms	PASS
T-12	Memory check	Flash 45.7%, RAM 57.6% (both < 100%)	PASS
T-13	Startup banner	Configuration banner printed once at init	PASS
T-14	Same hardware reuse	Report and implementation match the same Lab 5 wiring diagram	PASS

Table 7: Test execution results – Lab 6

All 14 test scenarios passed. The most informative tests were the spike-rejection and weighted-smoothing scenarios, because they directly validate the analogue-conditioning objective of Part 2.

4.3 Simulation Screenshots

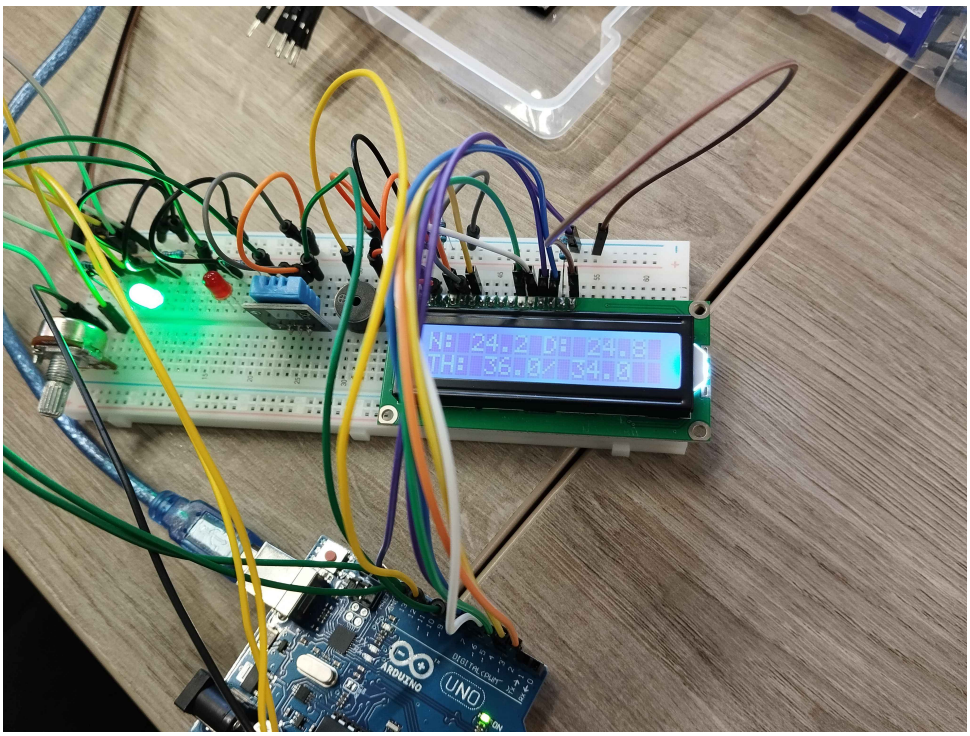


Figure 10: Wokwi simulation: normal operation with filtered values shown on LCD

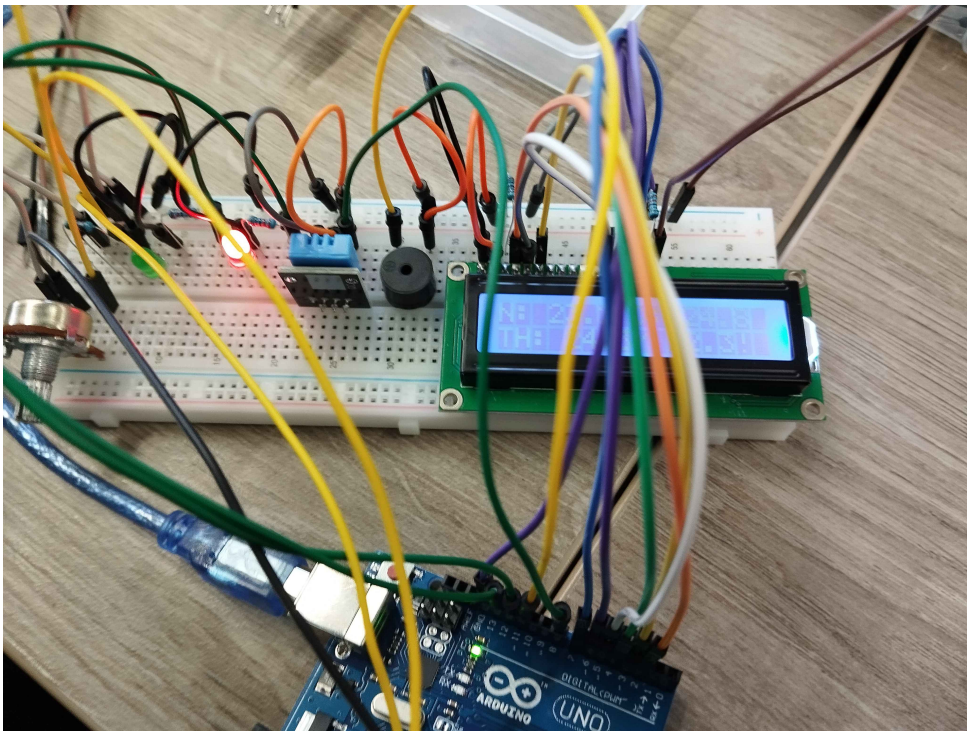


Figure 11: Wokwi simulation: filtered over-threshold condition using the same hardware indicators

4.4 Representative Serial Output

The following output is representative of a Wokwi simulation run in which the NTC channel is disturbed by noise and then rises above the reference threshold:

```
Lab6 - Part2 analog conditioning monitor
Same Wokwi diagram as lab5: NTC(A0), DHT11(D8), LCD, LEDs, buzzer, POT(A1)
Conditioning chain: saturation[-20.0.. 80.0] -> median(3) -> weighted average[50,25,1
Alerts: hysteresis around setpoint from potentiometer, confirmation=2 samples
Periods: acquisition=40ms reporting=500ms
---------------------------------------------------------------
t=500ms
  NTC adc= 512 V=2.50 Traw= 25.42C sat= 25.42 med= 25.40 avg= 25.36 cand=0 stable=0
  DHT valid=1 fresh=1 Traw= 24.00C sat= 24.00 med= 24.00 avg= 24.00 cand=0 stable=0
  TH set= 27.5C on>= 28.5C off<= 26.5C pot= 512 SYS=0
t=1000ms
  NTC adc= 447 V=2.18 Traw= 27.14C sat= 27.14 med= 27.10 avg= 26.48 cand=0 stable=0
  DHT valid=1 fresh=0 Traw= 24.00C sat= 24.00 med= 24.00 avg= 24.00 cand=0 stable=0
  TH set= 27.8C on>= 28.8C off<= 26.8C pot= 530 SYS=0
t=1500ms
  NTC adc= 430 V=2.10 Traw= 28.92C sat= 28.92 med= 28.90 avg= 28.11 cand=1 stable=1
```

```
DHT valid=1 fresh=0 Traw= 24.00C sat= 24.00 med= 24.00 avg= 24.00 cand=0 stable=0
TH set= 27.8C on>= 28.8C off<= 26.8C pot= 530 SYS=1
```

The trace highlights the Part 2 pipeline directly: each report shows the raw value, saturated value, median output, and weighted output. The optional alert only changes after the filtered value is confirmed by the reused validator.

4.5 Performance Analysis

- **Update latency**: New raw values enter the conditioning chain every 40 ms, so the displayed and filtered values satisfy the required sub-100 ms granularity.
- **CPU load**: The extra median and weighted-average calculations add only a few arithmetic operations per cycle and remain lightweight for Arduino Uno.
- **Float formatting**: `dtostrf()` remains necessary for AVR output formatting and dominates the reporting cost more than the filters themselves.
- **DHT11 refresh behaviour**: The DHT library does not provide a fresh measurement on every 40 ms cycle, so the last valid value is intentionally reused while preserving the acquisition rhythm.
- **Conditioning overhead**: The analogue filters require only small fixed-size buffers (3 and 4 samples), so they are appropriate for the 2 KB SRAM limit.

4.6 Testing Phase Conclusions

All 14 test scenarios passed. The STDIO trace format now exposes every conditioning stage, which makes debugging much easier than looking only at a final alert state.

General Conclusions

Laboratory Work #6 achieved all stated objectives. Using exactly the same hardware arrangement as Lab 5, the application was adapted for Part 2 so that the main focus became analogue signal conditioning rather than binary thresholding.

Key outcomes:

1. **Same hardware, different task**: The Lab 5 Wokwi/PlatformIO setup was preserved, proving that the assignment change is almost entirely in software.
2. **Dual-sensor acquisition**: The analogue NTC channel and digital DHT11 channel were integrated again behind clean C++ interfaces.

3. **Analogue conditioning pipeline**: The new `AnalogConditioner` module demonstrates the complete Part 2 chain: saturation, median filtering, and weighted averaging.
4. **Observable intermediate states**: STDIO output exposes raw, saturated, median, and weighted values, making the conditioning effect directly visible.
5. **Cooperative multi-rate loop**: The same 40 ms acquisition and 500 ms reporting structure remains simple and efficient for Arduino Uno.
6. **Modular architecture**: The project keeps the same clean layered structure while extending only the signal-processing layer with the new analogue filter module.

Possible improvements:

- Add an exponential moving average (EMA) and compare it experimentally with the weighted-average stage.
- Log the intermediate stages to CSV for plotting in Arduino Serial Plotter or an external tool.
- Replace the simple weighted average with an adaptive filter for faster response during rapid temperature changes.
- Generate new dedicated diagrams or screenshots that visualise the analogue pipeline stages more explicitly.

Conformance with UTM Evaluation Criteria

1. **Technological analysis & context**: Covered in Domain Analysis.
2. **Architectural design & HW–SW explanations**: Covered through architecture figures, pipeline table, and electrical description.
3. **Project structure & modular implementation**: Covered in Project Directory Structure and Software Module Descriptions.
4. **Hardware component description**: Each component is documented with role and reuse rationale.
5. **Software module description**: Each module is listed and explained with its responsibility.
6. **Functional implementation**: Requirements and test tables demonstrate implementation completeness.

7. **STDIO usage**: The `printf` pipeline through `StdioBridge` is detailed and representative output is included.
8. **Results presentation**: Diagram exports and simulation photos are included as visual evidence.
9. **Report documentation**: Report includes analysis, design, implementation, testing, conclusions, bibliography, AI usage note, and source-code annex.
10. **Optional extra functionality**: Bonus feature preserved and documented: potentiometer-controlled dynamic threshold reference (A1).

Note on AI Tool Usage

AI-assisted tools were used to support coding productivity and report drafting:

- **GitHub Copilot** – code suggestions, B-model implementation, analogue filter structuring, DHT11 fallback pattern.
- **LLM assistance** – technical writing structure, LaTeX formatting, and language polishing.

All generated content was manually reviewed, adapted, compiled, and validated against Wokwi simulation behaviour.

Bibliography

1. Horowitz, P., Hill, W. *The Art of Electronics*, 3rd ed. Cambridge University Press, 2015.
   (NTC thermistor model, voltage-divider circuits, basic filtering concepts.)
2. Adafruit Industries. *DHT Sensor Library Documentation.*
   `https://github.com/adafruit/DHT-sensor-library`
3. Wokwi Simulator Documentation.
   `https://docs.wokwi.com/`
4. PlatformIO Documentation.
   `https://docs.platformio.org/en/latest/`
5. AVR-libc Documentation – `dtostrf`, `fdev_setup_stream`.
   `https://www.nongnu.org/avr-libc/user-manual/`

6. Arduino Official Reference – `analogRead`, `tone`, `LiquidCrystal`.
   `https://www.arduino.cc/reference/en/`

7. Lab 6 Theory and Task Notes – signal conditioning, median filtering, and weighted averaging.
   Technical University of Moldova, 2026.

Annex – Source Code

The full source code is hosted on GitHub:

Repository URL:
`https://github.com/Ekkusuu/embedded-systems-repo/tree/main/lab6`

This repository contains all source files under `lab6/src/`, the PlatformIO configuration (`platformio.ini`), the Wokwi simulation files (`diagram.json`, `wokwi.toml`), and the report sources.

PlantUML diagram source files reused from the previous lab structure are stored in: `lab6/report/plantuml/`

Complete module listing:

- `src/main.cpp` – entry point, object instantiation, `setup()`/`loop()`
- `src/config/AppConfig.h` – all compile-time constants
- `src/sensors/NtcSensor.h/.cpp` – analogue NTC thermistor driver
- `src/sensors/DhtSensor.h/.cpp` – DHT11 digital sensor wrapper
- `src/signal/AnalogConditioner.h/.cpp` – saturation + median + weighted average
- `src/signal/BinaryThresholdFilter.h/.cpp` – reused alert validator
- `src/peripherals/LedController.h/.cpp` – green/red LED pair driver
- `src/peripherals/BuzzerController.h/.cpp` – passive buzzer driver
- `src/io/StdioBridge.h/.cpp` – `fdev_setup_stream` UART bridge
- `src/io/LcdDisplay.h/.cpp` – 16×2 LCD wrapper
- `src/app/MonitorApp.h/.cpp` – application orchestration